

LAB 6 – Assignment

Name: Krish Singh

Roll: AID25072

Class: AID-G sem 1

Program 1:

```
class Employee{
    int empId;
    String name;

    Employee(int id,String n) {
        empId = id;
        name = n;
        System.out.println("Employee constructor executed");
    }
}
```

Program 2:

```
class Manager extends Employee{
    String department;
    Manager(int id, String n, String dept){
        super(id,n);
        department=dept;
        System.out.println(("Manager constructor executed"));
    }
    public static void main(String[] args){
        Manager m = new Manager(101,"Ravi","HR");
        System.out.println("ID:"+m.empId);
        System.out.println("Name:"+m.name);
        System.out.println("Department:"+m.department);
    }
}
```

Output:

```
/usr/java/jdk-17-oracle-x64/bin/java -javaagent:/home/krish/.local/s
Employee constructor executed
Manager constructor executed
ID:101
Name:Ravi
Department:HR

Process finished with exit code 0
```

Question Answers:

1. Which constructor executed first?

Ans: The Employee constructor is executed first. Java automatically calls the parent class constructor first using `super(id,n);`

2. Did you explicitly create an Employee object?

Ans: No, I did not explicitly create a Employee object. The Employee constructor runs automatically as Manager extends Employee.

3. Why are `empId` and `name` accessible in Manager?

Ans: `empId` and `name` are accessible in Manager as manager inherits properties of Employee (Manager extends Employee)

4. Who initializes employee details — Employee or Manager?

Ans: Employee initializes employee details. The line `super(id,n)` calls Employee constructor and, inside that constructor employee sets `id` and `name`. Manager only initializes `department`.

Program 3:

```
class Manager extends Employee{
    String department;
    Manager(int id, String n, String dept){
        department=dept;
        System.out.println("Manager constructor executed");
    }
    public static void main(String[] args){
        Manager m = new Manager(101,"Ravi","HR");
        System.out.println("ID:"+m.empId);
        System.out.println("Name:"+m.name);
        System.out.println("Department:"+m.department);
    }
}
```

Output:

❗ Lab 6: build failed At 2/11/26, 11:41 AM with 1 error 866 ms

▼ 📄 Manager.java src 1 error

❗ constructor Employee in class Employee cannot be applied to given types; :3

```
java: constructor Employee in class Employee cannot be applied to given types;
  required: int,java.lang.String
  found:    no arguments
  reason: actual and formal argument lists differ in length
```

Questions (Based on Error)

1. What compilation error occurs?

Ans: Here a constructor mismatch error occurs. Java tries to call the parent constructor automatically, but it cannot find a constructor in Employee that takes zero parameters. Since only a parameterized constructor exists, compilation fails.

2. Why does Java not allow object creation here?

Ans: Java does not allow object creation here because Employee class has only parameterized constructor, it has no default constructor, When we remove `super(id,n)` Java automatically tries to insert `super()` but `employee()` (no argument constructor) does not exist, so Java cannot initialize the parent part of the object and hence compilation fails

3. Does Java automatically call parent constructor in all cases?

Ans: Yes, Java calls parent constructor in all cases. Automatic call happens, but only to `super()` that is to the default constructor with no arguments.

4. When does `super ()` become compulsory?

Ans: `super()` becomes compulsory when parent class has only parameterized constructors and there is no default constructor. In these cases we must explicitly call `super(arguments)`.

Program 4:

```
class Employee{  
    int empId;  
    String name;  
    Employee(){  
        System.out.println("Default constructor case");  
    }  
}
```

Output:

```
/usr/java/jdk-17-oracle-x64/bin/java -javaagent:/home/krish/.ld  
Manager constructor executed  
ID:0  
Name:null  
Department:HR  
  
Process finished with exit code 0
```

Questions

1. Does the program run now?

Ans: Yes the program runs now.

2. Which constructor is called implicitly?

Ans: Since we have not mentioned super(). Java implicitly calls the parents default constructor which is : Employee()

3. Why did the program work without writing super()?

Ans: The program worked without writing super() because, Java automatically calls parent constructor (default constructor with no arguments.)

4. What difference do you observe compared to Program 3?

Ans: The difference I observe is that even though we have passed no arguments it calls the default constructor with no parameters, the default Employee constructor applies default values to ID and Name.

Program 5:

```
class Manager extends Employee{
    String department;
    Manager(int id, String n, String dept){
        department=dept;
        System.out.println("Manager constructor executed");
    }
    public static void main(String[] args){
        Manager m1 = new Manager(101,"Ravi","HR");
        Manager m2 = new Manager(102,"Rohit","IT");
    }
}
```

Output:

```
/usr/java/jdk-17-oracle-x64/bin/java -javaagent:/home/krish/.l
Default constructor executed
Manager constructor executed
Default constructor executed
Manager constructor executed

Process finished with exit code 0
```

Questions

1. How many times is Employee constructor executed?

Ans: Employee constructor is executed 2 times. Once for each Manager object created.

2. How many times is Manager constructor executed?

Ans: Manager constructor is executed 2 times. Once for each object (m1 and m2).

3. Does constructor execution depend on class or object?

Ans: constructor execution depends on object. Constructor is executed with object creation regardless of how many objects belong to the same class.

4. Can constructor execution be skipped?

Ans: No constructors cant be skipped. When a object is created, its constructor must execute once to initialize the object. Also, in inheritance, the parent constructor is always executed before the child constructor.

Post-Lab Questions

1. Why must Employee constructor execute before Manager?

Ans: Employee constructor must execute before manager because Manager is subclass of Employee or simply Manager is a type of Employee. Since Employee is the parent class and Manager is the child class, the parent part of the object must be initialized before child adds its own data. In inheritance the base class is constructed first to ensure proper object initialization.

2. Why are constructors not inherited?

Ans: Constructors are used to initialize a specific class. They are not inherited because: A constructors name is the same as the class. A child class has a different name. Initialization of each class must be handled separately. So constructors are called not inherited.

3. What problem occurs if Employee has no constructor?

Ans: If Employee has no constructor then Java provides a default constructor automatically, and there is no problem. But if Employee has only parameterized constructor and no default constructor, then the child class must explicitly call it using `super(parameters)`. Otherwise, compilation error occurs (constructor mismatch error).

4. Why is `super()` always the first statement?

Ans: `super()` is always first statement because the parent class must be initialized before the child class. Java enforces this rule to maintain proper construction order. You cannot access parent members before the parent is constructed.

5. What real-world relationship does Employee–Manager represent?

Ans: Employee-Manager represents “IS-A” relationship. A manager is an Employee, but with additional responsibilities like handling departments.

6. Can Manager exist without Employee initialization?

Ans: No. Since Manager extends Employee, it must first initialize the Employee part of the object. Without parent initialization, the object would be incomplete.

7. Why are constructors executed automatically?

Ans: Constructors are executed automatically because object initialization is mandatory. When an object is created using new Java automatically calls the constructor to allocate memory, initialize instance variables, prepare the object for use. This ensures every object starts in valid state.